

Random maze generation by recursive division, and shortest-path search to the exit

Mohamed Ali Tewfik Hamlil

M1 Artificial Intelligence, Université Grenoble Alpes
Algorithmic Problem Solving — APP1

September 12, 2026

Abstract

A rectangular area of $W \times H$ cells is laid out as a maze by recursive division: a full wall across the longest dimension, exactly one door in it, and the same rule inside each of the two halves. This report states that as a graph problem, shows that the requirements force the maze to be a spanning tree of the grid graph, and derives a generation algorithm together with the data structure it runs on and a proof that what it returns is that spanning tree. Giving every door unit weight makes the length of a route the number of doors crossed, which is what the shortest-path search of Section 4 minimises and what the Manhattan and Euclidean distances are compared against as heuristic guidance. A Python implementation checks the claims on generated instances, and the closing section gives the size of the structure and the cost of both algorithms.

Contents

1	Problem formulation	2
1.1	Statement	2
1.2	From a drawing of walls to a graph	2
1.3	What the specification forces E to be	3
1.4	Constraints on the exit	3
1.5	The two problems, restated on the graph	4
1.6	Conventions	4
1.7	Scope of the deliverable	4
2	Generating the maze into a graph	5
2.1	What has to be stored	5
2.2	The node record	5
2.3	The region tree	6
2.4	One division contributes exactly one door	6
2.5	One division step	6
2.6	The algorithm	7
2.7	Why the result is a spanning tree	7
2.8	The graph that comes out	8
3	Encoding the distance between adjacent cells as 1	9
4	Shortest path to the exit, with distance as heuristic guidance	9
5	Testing with a Python implementation	9
6	Complexity and analysis of the results	9

1 Problem formulation

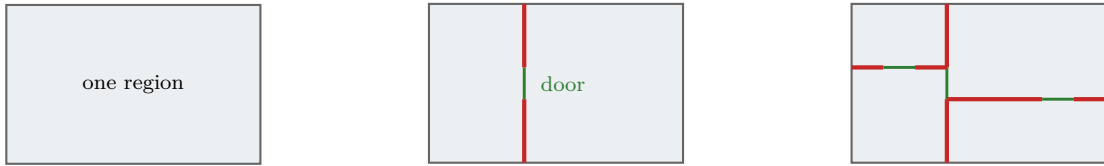
1.1 Statement

A rectangular area of $W \times H$ unit cells is to be laid out as a maze. The layout must be drawn at random, so that W and H do not determine it; the area has a single exit on its boundary; and it is entered at an arbitrary interior cell. Two problems follow, and both are to be settled algorithmically before any implementation and then analysed for cost:

1. given the width and the height of the initial area, generate a maze;
2. find the path from any position in the maze to the exit, which the specification states is unique and hence shortest.

The construction is prescribed. Divide the allocated space with a full wall across its longest dimension, leave exactly one door in that wall, and apply the same rule to each of the two spaces this creates. Figure 1 is the entire rule; everything the report derives follows from it.

“At random” constrains one half of the output and not the other. The cells are fixed by W and H : the same WH positions appear in every run, and their coordinates are public knowledge the moment the dimensions are. What must not be predictable is *which doors were kept* — the edge set. The requirement is therefore on the draws, not on the storage: the wall and door positions must come from a genuine random source, because any rule that derives them from the geometry alone (the midpoint, say) makes the maze a function of W and H and reconstructible by anyone holding those two numbers.



(a) A rectangular region, every cell reachable from every other. (b) One full wall across the longest side, pierced by exactly one door. (c) The same rule applied inside each of the two halves.

Figure 1: The construction rule. Each application turns one region into two regions of the same kind joined by a single door, which is what makes this a divide-and-conquer problem and the decomposition a binary tree.

1.2 From a drawing of walls to a graph

A maze drawn on paper is a picture of walls; what an algorithm can generate, store and search is a graph. The translation is the first decision of this report, and every later section rests on it.

Definition 1 (Area, cells, grid graph). Let $W, H \in \mathbb{N}^*$ be the width and the height of the allocated area, measured in unit cells. The cells are

$$V = \{(x, y) \in \mathbb{N}^2 : 0 \leq x < W, 0 \leq y < H\}, \quad |V| = WH,$$

and two cells are *adjacent* when they share a side, that is when $|x - x'| + |y - y'| = 1$. The *grid graph* $G_{\text{grid}} = (V, E_{\text{grid}})$ has the cells as vertices and the adjacent pairs as edges, so

$$|E_{\text{grid}}| = \underbrace{H(W - 1)}_{\text{horizontal}} + \underbrace{W(H - 1)}_{\text{vertical}} = 2WH - W - H.$$

Definition 2 (Maze). A *maze* on that area is a spanning subgraph $M = (V, E)$ with $E \subseteq E_{\text{grid}}$. An edge of E is a *door*: the two cells it joins can be walked between. A pair of $E_{\text{grid}} \setminus E$ is a *wall segment*. One distinguished boundary cell v_{exit} is the exit.

Figure 2 puts the two readings of the same object side by side. The vertex set is fixed by W and H alone; generating a maze therefore means nothing more than *choosing which subset of E_{grid} to keep*.

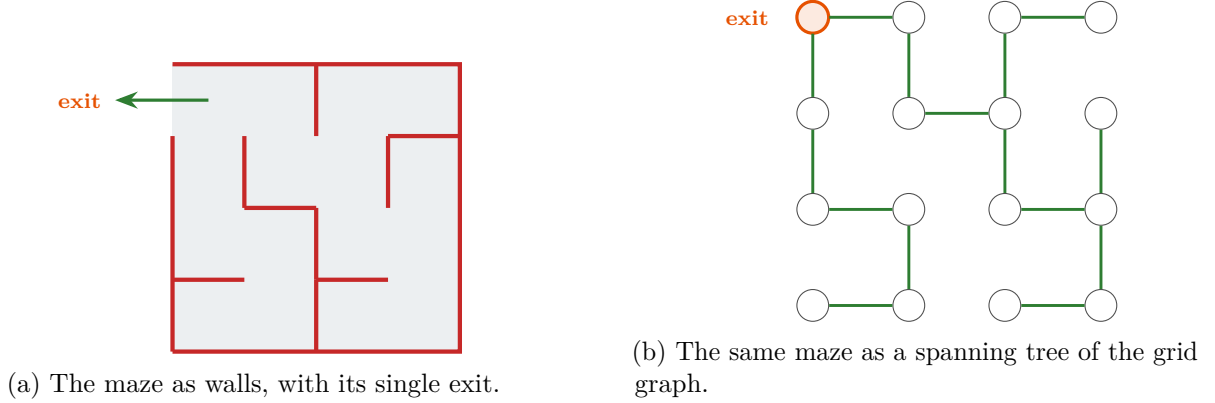


Figure 2: A maze is not a picture but a graph. Cells become vertices, doors become edges, and a wall is simply an edge of the grid graph that is absent. The two panels show the same 4×4 maze.

1.3 What the specification forces E to be

Three requirements follow from the statement.

- (R1) **Every cell reaches the exit.** The entry position is arbitrary, so any cell may be a starting point and each must have a route out: M is connected.
- (R2) **There is exactly one exit.** v_{exit} is a single cell on the boundary of the area, and the outer wall is closed everywhere else.
- (R3) **The route out does not fork.** The path to the exit is required to be unique, hence shortest; a cycle in M would give two distinct simple routes between the cells it passes through, so M is acyclic.

Connected and acyclic on a fixed vertex set is a spanning tree, which settles the shape of the answer before any algorithm is written.

Proposition 1 (The object to produce). *A subgraph $M = (V, E)$ of G_{grid} satisfies (R1) and (R3) if and only if it is a spanning tree of G_{grid} . It then has $|E| = WH - 1$ doors, and between any two cells there is exactly one simple path, which is therefore also a shortest one.*

That last clause is what makes the second problem well posed: asking for *the* path out is only meaningful because there is one. It is also the lever for the rest of the report — Section 2 shows that the construction rule produces such a tree *by construction* rather than by checking afterwards.

1.4 Constraints on the exit

Three questions about v_{exit} are worth settling before an algorithm is written, because two of them turn out to constrain nothing.

Must it be fixed before generation?

No. By Proposition 1 every cell reaches every other, so any cell designated afterwards is reachable from all of them. The exit is an opening in the *outer* boundary, and the outer boundary is not part of G_{grid} at all — the edges of G_{grid} run between cells, never out of the area. Generation and the choice of exit are independent, and Section 2 never consults it.

Must it be on the boundary?

Yes, but by definition rather than by necessity. An exit leads out of the area, so it is a boundary cell; this is requirement (R2) and a modelling decision, not something the algorithm enforces or could violate. The construction would accept any cell.

Must it have exactly one door?

No, and requiring it would be a mistake. Uniqueness of the route out comes from M being a tree, not from the exit's degree: whether v_{exit} has 1, 2, 3 or 4 doors, there is still exactly one simple path to it from anywhere. Worse, the constraint can be unsatisfiable — a spanning tree of G_{grid} is not obliged to have any of its leaves on the boundary, so demanding a degree-one boundary cell can rule out a maze the construction legitimately produced.

The exit is therefore a label applied to the finished tree, and the report treats it as one.

1.5 The two problems, restated on the graph

Problem 1 — Generation.

Given W , H , an exit cell v_{exit} and a source of random bits, produce a spanning tree M of G_{grid} , drawn at random so that no two runs need agree, together with the structure that records how it was built.

Problem 2 — Escape.

Given M and any start cell $s \in V$, return the unique simple path from s to v_{exit} .

Problem 1 is solved in Section 2. Problem 2 needs a notion of length before it can be measured: Section 3 gives every door weight 1, and Section 4 runs the shortest-path search that weight makes meaningful, guided by the Manhattan and Euclidean distances.

1.6 Conventions

Symbol	Meaning
W, H	width and height of the area, in cells
$n = WH$	number of cells, the size of the input
(x, y)	cell in column x (left to right) and row y (top to bottom)
$G_{\text{grid}}, E_{\text{grid}}$	the grid graph and its edges: every adjacency, all doors open
$M = (V, E)$	the maze produced: a spanning tree of G_{grid}
v_{exit}	the single exit cell, on the boundary
$[a_0, a_1] \times [b_0, b_1]$	a region: columns $a_0..a_1$, rows $b_0..b_1$

1.7 Scope of the deliverable

The specification fixes the contents and not only the topic: a data structure with a statement of what it holds, an algorithm for each of the two problems, the size of that structure and the cost of both algorithms. The table below is the report's own checklist against that list.

Required	Where	State
Data structure, and what it stores	Section 2	done
Maze generation algorithm	Section 2	done
Cost of a door, and what a path costs	Section 3	to write
Path-finding algorithm	Section 4	to write
Implementation (optional in the specification)	Section 5	to write
Size of the structure, complexity of both algorithms	Section 6	to write

2 Generating the maze into a graph

Section 1 fixed the target: a spanning tree of G_{grid} , drawn at random. This section builds one. The structure comes first, because the algorithm is written against it, and the guarantee comes last, because it is the construction rule that supplies it.

2.1 What has to be stored

Two different things are worth keeping, and confusing them is the usual way to end up storing neither well.

The maze itself — which doors exist.

It is what a walker needs, and the only thing needed to answer Problem 2.

The way it was built — the region decomposition.

Which wall was put where, and in what order. It is the binary tree the divide-and-conquer rule produces for free, and the object whose depth the cost analysis of Section 6 is written in terms of.

2.2 The node record

The maze is a set of nodes, one per cell. A node holds

$$\text{Node} = \left\langle \underbrace{\text{value}}_{\text{its } (i,j) \text{ coordinates}}, \underbrace{\text{neighbours}}_{\text{the cells it has a door to}} \right\rangle,$$

and the neighbour list is *empty when the node is created*. A cell has four sides, so the list never holds more than four entries, and it grows only when a division puts a door on that node. An index $\langle P_i, P_j \rangle$ — a $W \times H$ array of references — maps a coordinate pair to its node in $O(1)$, so a division can reach the two cells it wants without searching. Figure 3 is one node before and after.

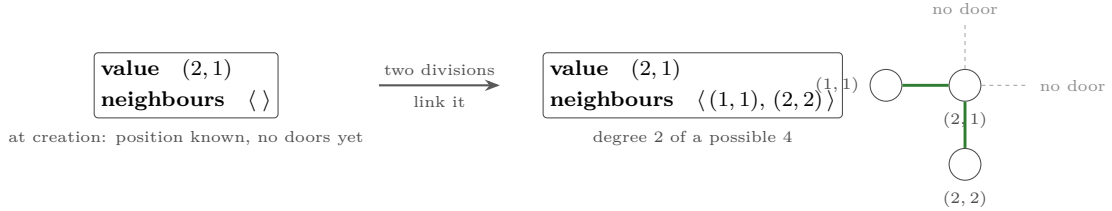


Figure 3: The node record. A node carries its coordinates and a list of the neighbours it has a door to; the list is empty when the node is made and is appended to only by a division whose door lands on that node. A cell has four sides, so the list never exceeds four entries, and in the finished maze the entries across all nodes number $2(WH - 1)$ — each door recorded once at each of its two ends.

Each door is recorded at both of its ends, so the finished maze holds $2(WH - 1)$ list entries for $WH - 1$ doors. That is the cost of being able to enumerate a cell's exits in $O(\text{deg})$ from the cell alone, which is what the search of Section 4 does at every step.

Remark. An alternative is to give each cell two links, to its right and lower neighbour, so that every *possible* door is owned by exactly one cell and none is stored twice. That trick exists to stop a dense grid of candidate edges being duplicated. It has nothing to save here: the construction below never represents a door that does not exist, so there is no duplication to avoid and no NIL to store for a wall.

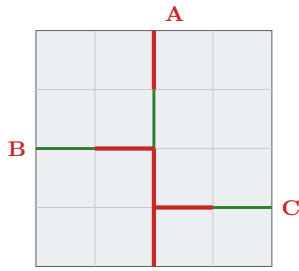
Remark. Storing the coordinates costs nothing in secrecy. The cells and their positions are fixed by W and H , and are known to anyone who knows the dimensions; what the randomness has to protect is the edge set, and a node’s *value* reveals nothing about which of its four sides became doors. The list does, but the list *is* the maze.

2.3 The region tree

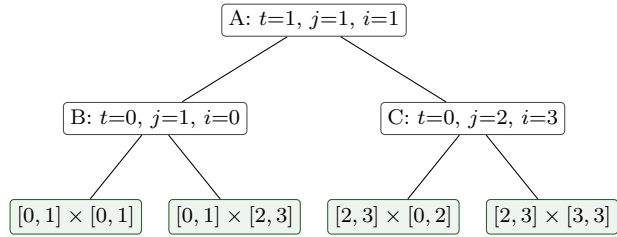
The second structure is a binary tree, one node per call of the recursion:

- an **internal node** records one division — its orientation t , the boundary j the wall sits on, the position i of its single door — and has the two regions the division created as its children;
- a **leaf** is a region reduced to one cell.

Figure 4 shows the first three divisions of a 4×4 area beside the top of the tree they build.



(a) The first three divisions of a 4×4 area.



(b) The top of the region tree it builds. The four boxes at the bottom are regions still to be cut, not cells.

Figure 4: Divisions and tree nodes are in bijection. An internal node records the orientation t , the wall position j and the door position i it drew, and its two children are the two regions it created; a leaf is a single cell. Carrying this 4×4 example down to its leaves gives 16 leaves and 15 internal nodes — one per door, matching the $WH - 1$ edges of Proposition 1.

2.4 One division contributes exactly one door

Everything below follows from a single observation. Take any door $e = (u, v)$ of the finished maze and walk down the region tree from the root. At each internal node, u and v either fall in the same child — and the walk continues — or they are separated, which happens precisely when e crosses that node’s wall. Since the leaves are single cells, they must be separated somewhere; and at the node that separates them, e is a crossing that survived, so e is that node’s door.

So each door belongs to exactly one internal node, and each internal node supplies exactly one door: internal nodes and doors are in bijection. A binary tree with WH leaves has $WH - 1$ internal nodes, and $WH - 1$ is the door count Proposition 1 demanded.

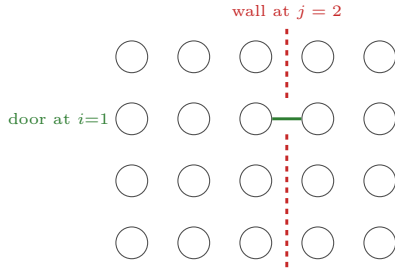
The construction reads that bijection forwards. Rather than opening every door and closing the ones a wall covers, a division *adds its own door and nothing else*. The maze is assembled edge by edge, one per call, and a wall is never drawn at all — it is simply the doors that were never added.

Remark. This is what makes the neighbour list a sound representation. A list cannot express “these two cells are not joined” except by omission, which is useless if the algorithm needs to remove links; here nothing is ever removed, so omission is exactly right.

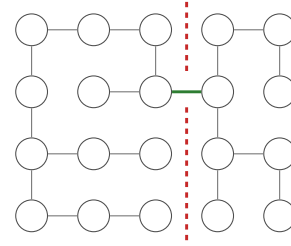
2.5 One division step

A call handles a region $[a_0, a_1] \times [b_0, b_1]$. If the region is a single cell there is nothing to divide and the call returns a leaf. Otherwise:

1. **Choose the orientation t .** The rule puts the wall across the longest dimension, so $t = 1$ (a vertical wall, cutting the column range) when $a_1 - a_0 > b_1 - b_0$, $t = 0$ (a horizontal wall, cutting the row range) when it is smaller, and a random bit decides a tie. The comparison is strict, so the dimension cut is the one of strictly greater extent; a dimension of extent 0 is never cut unless both are 0, which is the base case. This is what keeps the range $[a_0, a_1 - 1]$ that j is drawn from non-empty.
2. **Choose the wall position j ,** uniformly among the interior boundaries of the chosen dimension, and **the door position i ,** uniformly along the wall.
3. **Link the two cells the door falls on,** appending each to the other's neighbour list.
4. **Recurse** on the two regions, and hang them under the node recording $\langle t, j, i \rangle$.



(a) The call links one pair and stops. Every other node is still isolated; no crossing was examined, let alone removed.



(b) After both recursive calls return. Each half is a tree, and the door is still the only edge between them.

Figure 5: One division, with $t = 1$. The call draws the wall position j and the door position i , links (j, i) to $(j+1, i)$, and recurses on the two halves. Counting edges: 11 on the left, 7 on the right, one door, total $19 = 20 - 1$ on 20 cells — the tally of Proposition 2, assembled one edge per call.

2.6 The algorithm

Both cells a `Link()` touches lie in the region, since j and $j + 1$ are both in $[a_0, a_1]$ and i is in $[b_0, b_1]$, and they are adjacent — so every appended entry is a genuine edge of G_{grid} .

The exit plays no part in generation, for the reasons given in Section 1.4: the algorithm never reads it, and any boundary cell may be designated once the tree exists.

2.7 Why the result is a spanning tree

Proposition 2 (Correctness of Algorithm 1). *On a region of m cells, `Divide` leaves the subgraph induced on that region connected and acyclic, with exactly $m - 1$ doors. In particular `GenerateMaze` returns a spanning tree of G_{grid} .*

Proof. By induction on m . If $m = 1$ the region is one cell: no doors, connected and acyclic, $m - 1 = 0$, and the call adds nothing.

Let $m > 1$. The orientation rule cuts a dimension of extent at least 1, so j exists and the two sub-regions R_ℓ and R_r are non-empty, with $m_\ell + m_r = m$. The call adds the single door at i , whose two cells lie one in each sub-region, and then recurses; by the induction hypothesis the two calls leave their regions connected and acyclic with $m_\ell - 1$ and $m_r - 1$ doors. Neither recursive call can add an edge between the two sub-regions, because every `Link()` a call makes joins two cells of its own region.

Algorithm 1: Maze generation by recursive division.

Input: $W, H \geq 1$, the width and height of the area in cells.

Output: N , the set of nodes holding a spanning tree of G_{grid} ; r , the root of the region tree that built it.

```
1 Function GenerateMaze( $W, H$ ):
2    $N \leftarrow \{ \text{NewNode}(x, y) : 0 \leq x < W, 0 \leq y < H \}$ ; // every neighbour list empty
3    $r \leftarrow \text{Divide}(N, [0, W - 1], [0, H - 1])$ ;
4   return ( $N, r$ );

5 Function Divide( $N, [a_0, a_1], [b_0, b_1]$ ):
6   if  $a_0 = a_1$  and  $b_0 = b_1$  then return Leaf( $a_0, b_0$ );
7    $t \leftarrow \text{SplitOrientation}(a_1 - a_0, b_1 - b_0)$ ;
8   if  $t = 1$  then
9      $j \leftarrow \text{RandomIn}(a_0, a_1 - 1)$ ; // wall between columns  $j$  and  $j + 1$ 
10     $i \leftarrow \text{RandomIn}(b_0, b_1)$ ; // the one door, on row  $i$ 
11    Link( $N, (j, i), (j + 1, i)$ );
12     $\ell \leftarrow \text{Divide}(N, [a_0, j], [b_0, b_1])$ ;
13     $r \leftarrow \text{Divide}(N, [j + 1, a_1], [b_0, b_1])$ ;
14  else
15     $j \leftarrow \text{RandomIn}(b_0, b_1 - 1)$ ; // wall between rows  $j$  and  $j + 1$ 
16     $i \leftarrow \text{RandomIn}(a_0, a_1)$ ; // the one door, on column  $i$ 
17    Link( $N, (i, j), (i, j + 1)$ );
18     $\ell \leftarrow \text{Divide}(N, [a_0, a_1], [b_0, j])$ ;
19     $r \leftarrow \text{Divide}(N, [a_0, a_1], [j + 1, b_1])$ ;
20  return CutNode( $t, j, i, \ell, r$ );

21 Function Link( $N, u, v$ ):
22   append  $N[v]$  to  $N[u].\text{neighbours}$ ;
23   append  $N[u]$  to  $N[v].\text{neighbours}$ ;

24 Function SplitOrientation( $d_x, d_y$ ):
25   if  $d_x > d_y$  then return 1;
26   if  $d_x < d_y$  then return 0;
27   return RandomBit();
```

So the door at i is the only edge between R_ℓ and R_r . Joining two disjoint trees by a single edge yields a tree: connected, since any two cells are joined inside their own half or by crossing that door; acyclic, since a cycle would have to cross the wall an even, hence non-zero, number of times while only one crossing exists. The door count is $(m_\ell - 1) + (m_r - 1) + 1 = m - 1$.

Applying this to the root region gives a connected acyclic spanning subgraph of G_{grid} on all WH cells: a spanning tree, with $WH - 1$ doors. $\square$

The guarantee is therefore structural. Nothing is checked after the fact, no attempt is ever rejected and retried, and the uniqueness of the route out — the whole of Problem 2 — is a consequence of the shape of the recursion rather than of a property that had to be tested.

2.8 The graph that comes out

Generation returns the set N of nodes, each carrying its coordinates and the doors it ended up with, together with the root r of the region tree. Three readings of that one result, none of them a separate copy:

View	Held as	Used by
Adjacency	each node's neighbour list	the search of Section 4
Position	each node's <i>value</i>	the heuristics of Section 4
How it was built	the region tree rooted at r	the cost analysis of Section 6

The second row is what makes Section 4 possible at all: a heuristic that estimates the remaining distance to the exit needs the coordinates of the cell it is standing on, and they are in the node.

Figure 2b is the output of one 4×4 run, whose first three divisions are drawn in Figure 4a: 15 doors on 16 cells, one path between any two of them. What Section 3 adds is a length on those doors, so that “the path out” becomes a quantity a search can minimise.

3 Encoding the distance between adjacent cells as 1

To be written.

4 Shortest path to the exit, with distance as heuristic guidance

To be written.

5 Testing with a Python implementation

To be written.

6 Complexity and analysis of the results

To be written.